USB Driver Installer (`usb-driver-installer`)

A JVM library that installs the **WinUSB** driver for Android devices on **Windows**, so libusb-based apps (such as hosts built on `secureqt-sdk`) can open them. It bundles `wdi-simple.exe`, `zadig.exe` and a signed Android Open Accessory driver package, extracts them at runtime, and runs the installation behind a single UAC prompt.

- **Version:** `1.0.0`
- **Package:** `com.example.usbdriverinstaller.api` (public API)
- **Runs on:** any JVM 11+. **Installs drivers only on Windows**; on Linux/macOS every call is a safe no-op
- **Used by:** `secureqt-sdk` desktop hosts (see `secureqt-sdk/README.md` §3.5–3.7)

Table of Contents

1. Why it's needed
2. Getting the JAR
3. Adding it to your project
4. Quick start
5. Direct API: `IUsbDriverInstaller`
6. Managed API: `IUsbDriverInstallManager`
7. Results and states
8. Using it with `secureqt-sdk`
9. What happens during an install
10. Security considerations
11. Troubleshooting

---

1. Why it's needed

libusb on Windows can only open a USB device that has a **WinUSB, libusbK or libusb0** driver bound to it. Android phones in Android Open Accessory mode re-enumerate as **`18D1:2D00`** (accessory) or **`18D1:2D01`** (accessory + ADB). Windows doesn't bind a WinUSB driver to these IDs automatically, so `LibUsb.open()` fails with `-12 ERROR_NOT_SUPPORTED` until a driver is installed.

This library installs that driver from inside your app, so users don't need to download and run Zadig themselves.

Platform	Behaviour
Windows 10 / 11	Installs WinUSB (UAC prompt once per install)
Linux / macOS	`installWinUsb()` returns `NotWindows`; `isDriverAlreadyInstalled()` returns `true`. Nothing is executed

Because non-Windows calls are no-ops, the same app code and JAR can ship on all platforms.

---

2. Getting the JAR

There are two variants:

JAR	Size	Contains	Use when
`usb-driver-installer-standalone.jar` (fat JAR)	~20 MB	Library + tools + Kotlin stdlib + kotlinx-coroutines	Plain Java apps, or when you don't want to manage dependencies
`usb-driver-installer-1.0.0.jar` (thin JAR)	~16 MB	Library + tools	Kotlin apps that already depend on the stdlib and coroutines

Most of the size is the bundled Windows tools (`wdi-simple.exe` ~10 MB, `zadig.exe` ~5 MB).

From GitHub Releases: pushing an `sdk-v*` tag runs `.github/workflows/sdk-release.yml`, which publishes `usb-driver-installer-standalone.jar` next to the SDK JAR and AAR.

Build locally (from the repository root):

```
./gradlew :usb-driver-installer:standaloneJar
#   → usb-driver-installer/build/libs/usb-driver-installer-standalone-1.0.0-all.jar

./gradlew :usb-driver-installer:jar
#   → usb-driver-installer/build/libs/usb-driver-installer-1.0.0.jar

./gradlew :usb-driver-installer:test      # checks the bundled resources are present
```

---

3. Adding it to your project

Gradle (Kotlin DSL), fat JAR:

```
dependencies {
    implementation(files("libs/usb-driver-installer-standalone.jar"))
}
```

Gradle, thin JAR:

```
dependencies {
    implementation(files("libs/usb-driver-installer-1.0.0.jar"))
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")   // or newer
}
```

Same Gradle build (module dependency):

```
// settings.gradle.kts
include(":usb-driver-installer")

// consumer build.gradle.kts
dependencies { implementation(project(":usb-driver-installer")) }
```

Maven:

```
<dependency>
  <groupId>com.example.usbdriverinstaller</groupId>
  <artifactId>usb-driver-installer-standalone</artifactId>
  <version>1.0.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/libs/usb-driver-installer-standalone.jar</systemPath>
</dependency>
```

> If you shrink your app with ProGuard/R8, keep the `tools/**` resources. The library extracts them from the classpath at runtime and returns `ExtractionFailed` if they're missing.

---

4. Quick start

Kotlin

```kotlin
import com.example.usbdriverinstaller.api.DriverInstallResult
import com.example.usbdriverinstaller.api.UsbDriverInstallerFactory

fun ensureAoaDriver(): Boolean {
    val installer = UsbDriverInstallerFactory.getDefaultInstaller()

    if (installer.isDriverAlreadyInstalled(0x18D1, 0x2D00)) return true   // also true on Linux/macOS

    val result = installer.installWinUsb(
        vendorId = 0x18D1,
        productId = 0x2D00,
        deviceName = "Android Open Accessory"
    )
    when (result) {
        DriverInstallResult.Success -> println("Driver ready")
        DriverInstallResult.AccessDenied -> println("User declined the UAC prompt")
        else -> println("Install failed: ${result.message}")
    }
    return result.isSuccess
}
```

Java

```java
import com.example.usbdriverinstaller.api.DriverInstallResult;
import com.example.usbdriverinstaller.api.IUsbDriverInstaller;
import com.example.usbdriverinstaller.api.UsbDriverInstallerFactory;

public final class DriverSetup {
    public static boolean ensureAoaDriver() {
        IUsbDriverInstaller installer = UsbDriverInstallerFactory.getDefaultInstaller();

        if (installer.isDriverAlreadyInstalled(0x18D1, 0x2D00)) return true;

        // Java must pass every argument (Kotlin default values are not visible to Java)
        DriverInstallResult result =
                installer.installWinUsb(0x18D1, 0x2D00, "Android Open Accessory", 120_000L);

        System.out.println(result.getResultMessage());
        return result.isSuccessful();
    }
}
```

> Installing for VID `0x18D1` **also installs the driver for both `2D00` and `2D01`** in the same elevated run, so one call covers both accessory modes.

---

5. Direct API: `IUsbDriverInstaller`

Get an instance from the factory:

```kotlin
val installer = UsbDriverInstallerFactory.getDefaultInstaller()   // shared singleton
val other     = UsbDriverInstallerFactory.createInstaller()       // new instance
```

Method	Blocking?	UAC prompts	Description
`isWindows(): Boolean`	No	0	`true` on Windows
`isDriverAlreadyInstalled(vid, pid): Boolean`	~<1 s	0	Checks the registry (`HKLM\SYSTEM\CurrentControlSet\Enum\USB\VID_xxxx&PID_xx` for a `winusb` / `libusbk` / `libusb0` service. Always `true` off Windows
`installWinUsb(vid, pid, deviceName, timeoutMs = 120 000)`	**Yes**	**1**	Full install: removes conflicting drivers, runs `wdi-simple`, adds th bundled INF package, rescans and restarts the device. Returns a `DriverInstallResult`
`installWinUsbAsync(...)`	No	1	Same, as a `CompletableFuture<DriverInstallResult>` (Java friendly)
`suspend installWinUsbSuspend(...)`	Suspends	1	Same, runs on `Dispatchers.IO`
`launchZadigGui(timeoutMs = 300 000): Boolean`	**Yes**	1	Opens the bundled Zadig (List All Devices on, WinUSB preselected for manual selection. `true` if Zadig exits with code 0
`launchZadigGuiAsync(...)` / `suspend launchZadigGuiSuspend(...)`	No	1	Async variants
`cleanupConflictingDrivers(vid, pid): Boolean`	Yes (≤30 s)	1	Removes Samsung `ssudbus` / CDC `usbser` OEM drivers that clair this PID
`rescanAndRestartDevice(vid, pid): Boolean`	Yes (≤30 s)	**2**	`pnputil /scan-devices` + `/restart-device`
`extractTools(dir): Boolean`	Yes	0	Copies `wdi-simple.exe`, `zadig.exe`, the driver binaries and the INF/CAT/CER package into `dir` (for manual or offline installs)

> **Don't chain `cleanupConflictingDrivers()` → `installWinUsb()` → `rescanAndRestartDevice()`.** `installWinUsb()` already does the cleanup and the restart inside its one elevated script. Calling the other two adds three more UAC prompts. Use them only on their own, for example from a "Repair" button.

Never call the blocking methods on a UI thread. They wait for the user to answer the UAC prompt. Use `installWinUsbSuspend` (Kotlin) or `installWinUsbAsync` (Java), or call them from a background thread.

---

6. Managed API: `IUsbDriverInstallManager`

For apps with a UI, the manager wraps the installer in a small state machine. It:

- exposes progress as a `StateFlow` (Compose) or through listener callbacks (Swing/JavaFX/Java)
- runs installs on a background coroutine
- **attempts each VID:PID only once per session**, so a USB hot-plug loop can't re-prompt UAC endlessly
- maps failures to user-friendly messages

```kotlin
val manager = UsbDriverInstallerFactory.createManager()   // uses the default installer
```

Member	Description
`driverInstallState: StateFlow<DriverInstallState>`	Current state (see §7)
`onDeviceNeedsDriver(vid, pid, name)`	Starts an install unless one is already running or this VID:PID was tried this session. Returns immediately
`retryInstall(vid, pid, name)`	Clears the "already tried" mark for this device and installs again (for a "Retry" button)
`suspend waitForInstall(timeoutMs = 45 000): Boolean`	Waits for the running install. `true` if it ended `Installed`
`waitForInstallBlocking(timeoutMs)`	Blocking version for Java
`launchZadig()`	Opens the Zadig GUI in the background
`dismiss()`	Resets the state to `Idle` (close your dialog)
`resetAttempts()`	Forgets all "already tried" marks
`addListener(l)` / `removeListener(l)`	Callback API; `addListener` immediately delivers the current state

> `waitForInstall` returns right away if nothing is installing. Call `onDeviceNeedsDriver` first, then wait. The default timeout is 45 s; a user who is slow to answer UAC can exceed that while the install keeps running. Pass a larger timeout if needed.

Compose Desktop example

```
@Composable
fun DriverDialog(manager: IUsbDriverInstallManager) {
    when (val s = manager.driverInstallState.collectAsState().value) {
        is DriverInstallState.Idle -> Unit
        is DriverInstallState.Installing -> AlertDialog(
            onDismissRequest = {},
            title = { Text("Installing USB driver") },
            text = { Text("Approve the Windows security prompt to install the driver for ${s.deviceName}.
") },
            confirmButton = {}
        )
        is DriverInstallState.Installed -> Text("Driver installed for ${s.deviceName}")
        is DriverInstallState.Failed -> AlertDialog(
            onDismissRequest = manager::dismiss,
            title = { Text("Driver installation failed") },
            text = { Text(s.reason) },
            confirmButton = { Button({ manager.retryInstall(s.vendorId, s.productId, s.deviceName) }) { T
ext("Retry") } },
            dismissButton = { Button(manager::launchZadig) { Text("Open Zadig") } }
        )
    }
}
```

Java / Swing example

```
IUsbDriverInstallManager manager = UsbDriverInstallerFactory.createManager();

manager.addListener(state -> SwingUtilities.invokeLater(() -> {
    if (state instanceof DriverInstallState.Installing) {
        statusLabel.setText("Installing driver… approve the Windows prompt");
    } else if (state instanceof DriverInstallState.Installed) {
        statusLabel.setText("Driver installed");
    } else if (state instanceof DriverInstallState.Failed) {
        statusLabel.setText(((DriverInstallState.Failed) state).getReason());
    }
```

```
}));

// When LibUsb.open() fails with ERROR_NOT_SUPPORTED (-12) for an AOA device:
manager.onDeviceNeedsDriver(0x18D1, 0x2D00, "Android Open Accessory");
boolean ok = manager.waitForInstallBlocking(120_000L);   // call off the EDT
```

Listener callbacks run on a background thread. Switch to your UI thread before touching UI components.

---

7. Results and states

`DriverInstallResult` (returned by `installWinUsb*`)

Result	isSuccess	Cause
`Success`	✅	Installed, or already present
`AccessDenied`	❌	UAC declined (exit `1223`), or admin rights required (`-3`, `-15`)
`DeviceNotFound`	❌	Device not connected during install (`-4`)
`UnsignedDriverBlocked`	❌	Driver signature policy blocked it (`-19`)
`ExtractionFailed`	❌	Bundled `tools/` resources missing from the classpath
`NotWindows`	❌	Called on Linux/macOS. Treat as "nothing to do"
`Failure(exitCode, message)`	❌	Anything else. `message` holds the last 500 characters of the installer log. Exit code `2` means a serial (`usbser`) driver was picked instead of WinUSB

Kotlin: `result.isSuccess`, `result.message`. Java: `result.isSuccessful()`, `result.getResultMessage()`; singleton results are compared with `result == DriverInstallResult.Success.INSTANCE`.

`DriverInstallState` (emitted by the manager)

```
Idle ──onDeviceNeedsDriver()──► Installing ──success──► Installed ──(3 s)──► Idle
                                     │
                                     └──failure──► Failed ──retryInstall()──► Installing
                                                      └──dismiss()──► Idle
```

State	Fields
`Idle`	–
`Installing`	`deviceName`, `vendorId`, `productId`
`Installed`	`deviceName`, `vendorId`, `productId` (returns to `Idle` after 3 s)
`Failed`	`deviceName`, `reason` (user-friendly), `vendorId`, `productId`

A `NotWindows` result goes straight back to `Idle`.

---

8. Using it with `secureqt-sdk`

The installer has no dependency on the SDK. You connect them in your host code: when opening the AOA device fails because no driver is bound, install the driver, then scan the bus and open the device again.

```kotlin
import com.example.usbdriverinstaller.api.UsbDriverInstallerFactory
import org.usb4java.LibUsb

val installer = UsbDriverInstallerFactory.getDefaultInstaller()

fun onOpenFailed(vid: Int, pid: Int, error: Int): Boolean {
    if (!installer.isWindows()) return false
    if (error != LibUsb.ERROR_NOT_SUPPORTED && error != LibUsb.ERROR_NOT_FOUND) return false
    // Driver present but not bound yet → just retry
    if (installer.isDriverAlreadyInstalled(vid, pid)) return true

    val result = installer.installWinUsb(vid, pid, "Android Open Accessory")   // one UAC prompt
    if (result.isSuccess) Thread.sleep(1500)   // let Plug and Play bind the driver
    return result.isSuccess                    // caller re-scans the bus and retries
}
```

`secureqt-sdk/README.md` §3.5 has a complete `WindowsHostPlatform` built this way. It also handles `adb` holding the device and Plug and Play binding delays.

Which IDs to install for:

Device state	VID:PID	Needed for
AOA accessory	`18D1:2D00`	Opening the accessory (USB debugging off)
AOA accessory + ADB	`18D1:2D01` (interface 0)	Opening the accessory (USB debugging on)
Phone in normal mode	e.g. `04E8:6860` (Samsung)	Only if the AOA switch fails because no interface has a WinUSB-compatible driver. **This hides the phone from Explorer/MTP** while the driver is bound

One call for `18D1:2D00` (or `2D01`) covers both AOA IDs.

---

9. What happens during an install

`installWinUsb()` runs this sequence. Steps 3–7 run inside **one** elevated batch script (one UAC prompt):

1. Extracts `wdi-simple.exe` into `%TEMP%\sqt_wdi_*`.
2. Extracts the driver binaries (amd64/x86/arm64 co-installers, libusbK/libusb0) and the INF/CAT/CER package into `%TEMP%\sqt_driver_*`.
3. Removes OEM drivers that claim the same PID and are Samsung `ssudbus` or CDC `usbser`.
4. Runs `wdi-simple.exe -t 0` (WinUSB) for the requested VID:PID. For VID `18D1`, also for `2D00` and `2D01`.
5. Imports `aoa_driver.cer` and the CAT signer certificates into `LocalMachine\TrustedPublisher` (and `Root`), then `pnputil /add-driver … /install` for the bundled `Android_Accessory_Interface*.inf`.
6. `pnputil /scan-devices`, then `/restart-device` for the target ID, its `MI_00` interface, `18D1:2D00` and `18D1:2D01`.
7. Writes everything to `wdi_install.log` in the staging folder. The library also prints the log to stdout with the prefix `[ZadigDriverInstaller][wdi-log]`.

The exit code is then mapped to a `DriverInstallResult` (§7).

---

10. Security considerations

Read this before shipping to managed or corporate PCs:

- **Certificates are added to the machine trust stores.** The install imports `aoa_driver.cer` into `LocalMachine\Root` and `LocalMachine\TrustedPublisher`. `wdi-simple` also creates and trusts its own self-signed certificate for the generated driver. Tell IT/security teams about this; some policies forbid it or remove it.
- **Elevated PowerShell/cmd runs with `-ExecutionPolicy Bypass`.** Endpoint protection may flag or block this. If so, pre-install the driver centrally (run `extractTools()` and deploy the INF with `pnputil` through your software deployment tool).
- **Conflicting OEM drivers are deleted** (`ssudbus`, `usbser`) when they match the PID. This can affect other tools that rely on those drivers (for example Samsung modem/serial features).
- **Temporary files** are created under `%TEMP%` and marked delete-on-exit; folders that still contain files may stay behind.

---

11. Troubleshooting

Symptom	Cause / fix
`AccessDenied`	User clicked **No** on UAC, or the account can't elevate. Retry and click **Yes**, or run once as an administrator
`UnsignedDriverBlocked`	Windows driver signature policy (or Secure Boot policy) blocked it. Deploy a properly signed driver through IT
`DeviceNotFound`	Cable unplugged, or the phone left AOA mode during install. Reconnect and switch to AOA again before installing
`ExtractionFailed`	`tools/**` resources stripped by shrinking/repackaging, or `%TEMP%` not writable
`Failure(2, …)`	A serial driver was picked instead of WinUSB. Use `launchZadigGui()` and pick **WinUSB** by hand
`Success`, but `LibUsb.open` still returns `-12`	Plug and Play hasn't finished binding. Wait 1–2 s and retry; unplug and replug if it continues
Works on one USB port, not another	Windows can create a new device instance per port on some systems. Install again on that port
Manager does nothing on a second `onDeviceNeedsDriver`	By design: one attempt per VID:PID per session. Use `retryInstall()` or `resetAttempts()`
Need to undo it	Device Manager → the device → **Uninstall device** and tick **Delete the driver software**, or `pnputil /enum-drivers` then `pnputil /delete-driver oemNN.inf /uninstall`

Project layout

```
usb-driver-installer/
├── build.gradle.kts                jar + standaloneJar (fat JAR) tasks
└── src/main
    ├── kotlin/com/example/usbdriverinstaller
    │   ├── api/                    Public API: IUsbDriverInstaller, IUsbDriverInstallManager,
    │   │                           UsbDriverInstallerFactory, DriverInstall{Result,State,Listener},
    │   │                           DriverType
    │   └── core/                   ZadigDriverInstallerImpl, DriverInstallManagerImpl,
```

```
│                                ResourceExtractor
└── resources/tools
    ├── wdi-simple.exe           libwdi command-line installer
    ├── zadig.exe                Zadig GUI
    ├── usb_driver/              WinUSB/libusbK/libusb0 co-installers
    │                            (amd64, x86, arm64)
    └── driver_pkg/              Android_Accessory_Interface*.inf/.cat
                                 + aoa_driver.cer
```